| GROUP 9

Customer Complaint Escalation Prediction

Using Machine Learning

Raihan- 2303
Zaid- 2195
Wassey- 0661
Affan- 2194
Raiyan- 2241

INTRODUCTION

The Problem

The problem

Shops get thousands of complaints. Agents handle them one by one, so urgent ones may wait too long.

Our idea

Teach a computer to read a complaint and say: Escalate (Yes) or Normal (No).

Why it helps

Urgent complaints reach senior staff fast. Customers stay happy.

DATA

The Data Set

100	15	9	61 / 39
records	columns	input features	Yes / No

Source

Customer Support Ticket Dataset from Kaggle (Suraj520).

Inputs

Age, gender, product, ticket type, subject, status, channel, wait hours, satisfaction rating.

TARGET VARIABLE

What Do We Predict?

- We predict Escalation_Required.
- Yes = 1: High or Critical priority.
- No = 0: Low or Medium priority.
- Models need numbers, so Yes/No becomes 1/0.

```
# Turn Yes/No into 1/0
y = df["Escalation_Required"].map(
    {"Yes": 1, "No": 0}
)

# Result
#   1 -> escalate to senior team
#   0 -> normal Level-1 support
```

Step 1: Look and Clean

- First check the size, types and summary.
- Count missing values and duplicates.
- Remove them so no ticket is counted twice.
- Our data: zero missing, zero duplicates.

```
df = pd.read_csv("tickets.csv")

print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
print(df.duplicated().sum())

df = df.drop_duplicates()
df = df.dropna()
```

Step 2: Make a New Feature

- The data has a reply time stamp.
- We turn it into one number: Response_Hours.
- It shows how long the customer waited.
- A long wait often means an angry customer.

```
df["First Response Time"] = pd.to_datetime(
    df["First Response Time"])

df["Response_Hours"] = (
    df["First Response Time"].dt.hour
    + df["First Response Time"].dt.minute
      / 60.0
).round(2)
```

Step 3: Text to Numbers

- Computers only understand numbers.
- LabelEncoder gives each category a code: 0, 1, 2...
- We save each encoder, so the app uses the same codes later.

```
categorical_columns = [
  "Customer Gender", "Product Purchased",
  "Ticket Type", "Ticket Subject",
  "Ticket Status", "Ticket Channel"]

encoders = {}
for col in categorical_columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
    encoders[col] = le
```

Step 4: Scale and Split

- Scaling puts all numbers on a similar range.
- Then split: 80% to train, 20% to test.
- Test data stays hidden to check real performance.
- stratify=y keeps the Yes/No ratio equal.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = \
    train_test_split(
        X_scaled, y,
        test_size=0.20,
        random_state=42,
        stratify=y)
```

Three Models We Compared

kNN

Votes of the 5 most similar past complaints.

AdaBoost

Boosting: models fix each other's mistakes.

Random Forest

Bagging: 100 trees vote together.

Why compare?

Different methods learn differently. We pick the best.

```
models = {
 "kNN": KNeighbors
   Classifier(
   n_neighbors=5),
 "Random Forest":
   RandomForest
   Classifier(
   n_estimators=100),
 "AdaBoost":
   AdaBoost
   Classifier(
   n_estimators=50)
}
```

Training and Testing Each Model

- Same loop for all 3 models.
- Fit on training data.
- Predict on unseen test data.
- Save the scores to compare.

```
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    prob = model.predict_proba(
        X_test)[:, 1]

    rec = recall_score(y_test, pred)
    f1  = f1_score(y_test, pred)
    auc = roc_auc_score(y_test, prob)
    cv  = cross_val_score(
        model, X_scaled, y, cv=5)
```

EVALUATION METRICS

How We Measure Success

Accuracy

Share of all complaints predicted correctly.

Precision

Of predicted escalations, how many were really urgent.

Recall (most important)

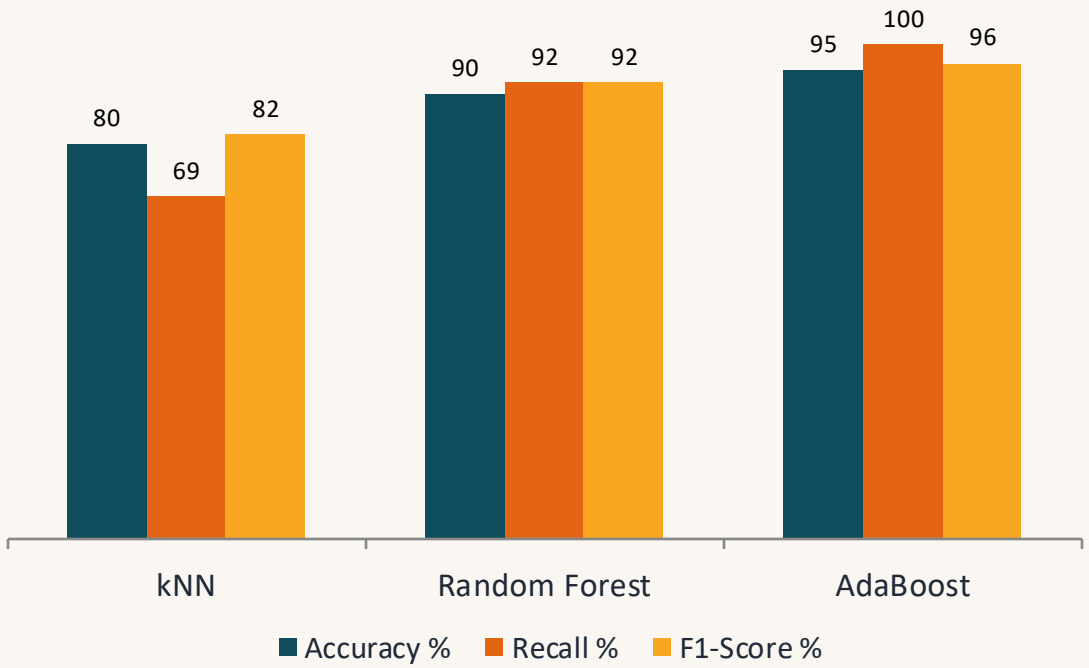
Of all urgent complaints, how many we caught. Missing one is costly.

F1-Score and ROC-AUC

F1 balances Precision and Recall. ROC-AUC shows how well the model separates Yes from No.

RESULTS

Results: Model Comparison



WINNER

AdaBoost

Accuracy 95%
Recall 100%
ROC-AUC 0.978
5-fold CV 93%

RESULTS ON 20 TEST TICKETS

Confusion Matrix: AdaBoost

	Predicted Yes	Predicted No
Actual Yes	13 TP	0 FN
Actual No	1 FP	6 TN

TP = urgent, caught TN = routine, correct
FP = false alarm FN = urgent, missed

Key findings

19 of 20 tickets correct.

Zero urgent complaints missed (FN = 0).

Only 1 false alarm.

Top factors: satisfaction rating and wait time.

APPLICATION

The Prediction App

- User types in the ticket details.
- Input is encoded and scaled like the training data.
- AdaBoost predicts Escalate or Not, with a risk %.
- Model, scaler and encoders are saved with joblib.

```
input_prepared = scaler.transform(
    input_data)

cls = best_model.predict(
    input_prepared)[0]
prob = best_model.predict_proba(
    input_prepared)[0][1] * 100

if cls == 1:
    print("YES - ESCALATE")
else:
    print("NO - Level-1 support")
print(f"Risk: {prob:.1f}%")
```

Conclusion

- AdaBoost is the best model: 95% accuracy, 100% recall.
- No urgent complaint was missed on the test data.
- Limit: only 100 records, so more data is needed to confirm.
- Next: more data, model tuning, a web app.

Thank you